Jay VanderZanden

2/20/2026

IT FDN 100 GP

Assignment 07

Class Objects and OOP

Introduction

Object oriented programming is a fundamental style of programming for organization and modularity in a program. Understanding how to create and use class objects is necessary to truly utilize a programming language. In this assignment I will discuss creating an Object class that is intended to be a parent class. Next I will discuss inheriting the parent class with another class object. Finally, I will discuss how to convert assignment 6 code to work with said object classes instead of lists of dictionaries.

Person Object Class

The first step of converting assignment 6 to work with objects is to create the desired parent class. The first step is to create the initialization function with the desired properties. I am going to want a first name and last name property for the person class. The code can be seen in figure 1.

```
# TODO Create a Person Class
class Person:  1 usage

    # TODO Add first_name and last_name properties to the constructor
    # creates properties
    def __init__(self, first_name: str = "", last_name: str = ""):
        self.first_name = first_name
        self.last_name = last_name
```

Figure 1

The next step is to create the getters and setters for the properties otherwise the class would be useless. To do this, I will be using decorators. The code is in figure 2.

```python
    # TODO Create a getter and setter for the first_name property
    # first_name getter
    @property  6 usages (4 dynamic)
    def first_name(self):
        return self.__first_name.title()

    # first_name setter
    @first_name.setter  5 usages (4 dynamic)
    def first_name(self, value: str):
        # check for numbers in first name value
        if value.isalpha() or value == "":
            # sets parameter value to first_name if valid
            self.__first_name = value
        else:
            raise ValueError("A first name should not contain numbers")
```

Figure 2

The @property decorator is the getter decorator. The function is going to be defined as the same of the property and it will take self as the parameter. All the getter needs to do is return the property. The double underscore is needed to reduce the risk of overwriting the property when creating objects. The .title() is a function that capitalizes the first letter. The setter will be defined with the name of the property and it will take two parameters, self and a string value that will become the first name. Since we do not want numbers in the first name, I will set up an if-else statement that will check if there is a number and if not, then it will set the value to the property otherwise it will raise a value error. The code for the last name property will be done the same just with last_name instead. The code for that can be viewed in figure 3.

```python
    # TODO Create a getter and setter for the last_name property
    # last_name getter
    @property  6 usages (4 dynamic)
    def last_name(self):
        return self.__last_name.title()
    # last_name setter
    @last_name.setter  5 usages (4 dynamic)
    def last_name(self, value: str):
        # check for numbers in last name value
        if value.isalpha() or value == "":
            # sets parameter value to last_name if valid
            self.__last_name = value
        else:
            raise ValueError("A last name should not contain numbers")
```

Figure 3

Lastly, I will redefine the string function. This is done with the code viewable in figure 4. All it will need to do is return a formatted string. The code for this can be viewed in figure 4.

```python
    # TODO Override the __str__() method to return Person data
    # overriding the __str__() method to return Person Data
    def __str__(self):
        return f"{self.first_name},{self.last_name}"
```

Figure 4

Student Object Class

With the person class made, I will now create a student class that inherits the person class. This will mean a student object will have the properties of the person object but have additional properties that are added within the class. For this assignment a student will not only contain a first and last name but also have a course name associated with it. This means I will need to create a property for the course name and create a getter and setter for that property. This is done just like the person class. So I will just be showing the code. To inherit the person class. When you create the Student class instead of just having the class name like the person class, you will need to pass the person class into the student class like a function. When writing the init function, you will also need the parameters for the first and last name, as well as the course name and within the function, you will need to use the init function from the parent class. The code can be viewed in figure 5.

```python
# TODO Create a Student class the inherits from the Person class
class Student(Person):  4 usages

    # creates properties
    # TODO call to the Person constructor and pass it the first_name and last_name data
    # TODO add a assignment to the course_name property using the course_name parameter
    def __init__(self, first_name: str = "", last_name: str = "", course_name: str = ""):
        super().__init__(first_name, last_name)
        self.course_name = course_name

    # TODO add the getter for course_name
    @property  6 usages (4 dynamic)
    def course_name(self):
        return self.__course_name.title()

    # TODO add the setter for course_name
    @course_name.setter  5 usages (4 dynamic)
    def course_name(self, value):
        self.__course_name = value

    # TODO Override the __str__() method to return the Student data
    def __str__(self):
        return f"{self.first_name},{self.last_name}, {self.course_name}"
```

Figure 5

With that, the object classes needed have been made.

Turning a List of Dictionaries to a list of objects

Since json files work with dictionaries, to read from a json file but work with student objects, I will need to convert the list of dictionaries from the json file into a list of objects. This will be done in the read from file function. Although not strictly necessary, I will first add a parameter that will be the object class I wish to use. I imagine making the object class a parameter is best practice and that is what I did. The line of code that is

commented out one line 138 is what you would use if not using a function parameter. Next, I will create a list variable that I wish to use to store the student object. Much like assignment 6, I will load the json file into a list variable. This list variable will be different than the one that I desire to return. Unlike assignment 6, instead of returning this variable, I will loop through this list variable. The first step in this loop is to create a student object variable and this will be done by calling the Class which will be denoted by (). The next step will be to assign each element of the dictionary to the necessary object property. When that is done you can then append the student object to the list that you want to return. After the exception block of the try, you can return this list. The code can be viewed in figure 6. Since the exception block of code is the same as assignment 6 and the return statement was mentioned above it will not be shown as it did not all fit on 1 window.

```
110     @staticmethod  1 usage
111     def read_data_from_file(file_name: str, student_type_object: Student):
112         """ This function reads data from a json file and loads it into a list of dictionary rows
113         then returns the list filled with student data.
114
115         ChangeLog: (Who, When, What)
116         RRoot,1.1.2030,Created function
117         Jay VanderZanden, 03/08/2026, converted function to work with objects
118
119         :param file_name: string data with name of file to read from
120         :param student_type_object: a reference to the Student class (object type Student)
121
122         :return: list
123         """
124         file = None
125         student_data: list = []
126         try:
127             # Get a list of dictionary rows from the data file
128             file = open(file_name, "r")
129             json_students = json.load(file)
130
131
132             # TODO replace this line of code to convert dictionary data to Student data
133             # Loops through each dictionary in the list loaded from the file
134             for student in json_students:
135
136                 # creates a object variable as type student from the parameter
137                 student_object = student_type_object()
138                 # student_object = Student()
139
140                 # assigns each property to the respective dictionary item
141                 student_object.first_name = student["FirstName"]
142                 student_object.last_name = student["LastName"]
143                 student_object.course_name = student["CourseName"]
144
145                 # assigns the object to the list passed into the function
146                 student_data.append(student_object)
```

Figure 6

Writing a List of Objects to a Json file

Since we are working with json files but we also want to work with class objects, we will need to convert objects back into a list of dictionaries to write the data to a json file. This is going to be done in the function to write to a file. The first step is to create a local

variable that is a list intended to take dictionaries. We are then going to loop through the list of objects that is passed into the function and assign each object's property to the necessary element of the dictionary that we create and then append that dictionary to the list. When that is done, the list of objects is converted to a list of dictionaries and we can proceed as was done in assignment 6. The new code can be viewed in figure 7.

```python
def write_data_to_file(file_name: str, student_data: list):
    """ This function writes data to a json file with data from a list of dictionary ro

    ChangeLog: (Who, When, What)
    RRoot,1.1.2030,Created function
    Jay VanderZanden, 03/08/2026, converted function to work with objects

    :param file_name: string data with name of file to write to
    :param student_data: list of dictionary rows to be writen to the file

    :return: None
    """
    file = None

    try:
        # TODO Add code to convert Student objects into dictionaries (Done)

        # creates a list of dictionaries to load onto a json file
        list_of_dictionaries: list = []

        #loopps through the list of objects to convert each element into a dictionary

        for student in student_data:
            # converts student object info into a dictionary
            student_json: dict = {"FirstName": student.first_name,
                                  "LastName": student.last_name,
                                  "CourseName": student.course_name}

            # appends the dictionary to the list of dictionaries variable
            list_of_dictionaries.append(student_json)

        # load desired file into write mode and assigns it to a variable
        file = open(file_name, "w")
```

Figure 7

Printing a List of Objects

Continuing with the trend of covering the changes to the functions from top to bottom, next is to cover the print function for printing the students in the list. The code for this is very similar to when working with dictionaries. Like assignment 6, we will loop through the list of objects passed into the function and only instead of having the syntax to work with a dictionary, we will instead work with objects (shocking I know). The code for which can be viewed in figure 8.

```python
    @staticmethod  2 usages
    def output_student_and_course_names(student_data: list):
        """ This function displays the student and course names to the user

        ChangeLog: (Who, When, What)
        RRoot,1.1.2030,Created function
        Jay VanderZanden, 03/08/2026, converted function to work with objects

        :param student_data: list of dictionary rows to be displayed

        :return: None
        """

        print("-" * 50)
        # loops through the list of objects to print each student object

        for student in student_data:

            # TODO Add code to access Student object data instead of dictionary data
            # calls the getter for the object for each property and formats for a print statement
            print(f"Student {student.first_name} {student.last_name} is enrolled in {student.course_name}")

            # dictionary print statement from the original
            # print(f'Student {student["FirstName"]} '
            #       f'{student["LastName"]} is enrolled in {student["CourseName"]}')

        print("-" * 50)
```

Figure 8

When you compare the commented out code to the code being used, you can see how similar the code is, with the only difference being how each property is referenced.

Adding objects from user input

The last block of code that needs changed to work with objects is in the function that handles the user input for a student. In this case, the input statements and their respective errors will be left untouched and I will add the code after. The first step is to create a variable that takes the parameter for the object class like done previously when reading the data from a file. Next, I will create a dictionary that will add the inputs for the students first, last, and course name. I will then use the student object variable created early and set it equal to each element in the dictionary referencing the pertinent properties. The last step is to append that object to the list passed into the function as a parameter. The code can be viewed in figure 9

```python
        try:
            student_first_name = input("Enter the student's first name: ")
            if not student_first_name.isalpha():
                raise ValueError("The last name should not contain numbers.")
            student_last_name = input("Enter the student's last name: ")
            if not student_last_name.isalpha():
                raise ValueError("The last name should not contain numbers.")
            course_name = input("Please enter the name of the course: ")

            # TODO Replace this code to use a Student objects instead of a dictionary objects

            # takes the parameter for the object class and assigns it to a local variable
            student_object = student_type_object()

            # student_object = Student()

            # I don't understand why this alone does not work
            # student_object.first_name = student_first_name
            # student_object.last_name = student_last_name
            # student_object.course_name = course_name

            # puts the inputs into a dictionary
            student = {"FirstName": student_first_name,
                       "LastName": student_last_name,
                       "CourseName": course_name}

            # converts the dictionary into a student object
            student_object.first_name = student["FirstName"]
            student_object.last_name = student["LastName"]
            student_object.course_name = student["CourseName"]
```

Figure 9

The code not pictured is the error handling except statements and of course the append code, however this code is the same as with assignment 6 thus I will leave the picture of this out. The line that is commented out just below the break point works, I just desired to handle the object as a parameter of the function instead of hard coding it. This same line of code could have been used in

Summary

Class objects increase the customizability of a program. It allows the user to create custom objects and define custom functions that are intended to work with these objects. Class Objects is a fundamental aspect of many programming languages and is necessary to truly utilize the full modularity and organization of a language.